

Actividad 1. Introducción al Aprendizaje Automático con Meta AI.

Prompt 1. ¿Cuáles son las diferencias fundamentales entre el Aprendizaje Supervisado, No Supervisado y por Refuerzo? Para cada paradigma, proporciona un ejemplo de caso de uso real en la industria y explica qué tipo de datos y objetivos requiere cada uno.

Las 3 ramas del Machine Learning se diferencian principalmente por el tipo de "retroalimentación" que reciben para aprender y el objetivo que buscan.

Tabla comparativa

Paradigma	¿Cómo aprende?	Datos que necesita	Objetivo principal	Analogía
Aprendizaje Supervisado	Con ejemplos etiquetados: "entrada → salida correcta"	Dataset con X = características, y = etiqueta/respuesta conocida	Predecir o clasificar nuevos casos	Estudiar con guía de respuestas
Aprendizaje No Supervisado	Sin etiquetas. Busca patrones ocultos por sí mismo	Solo X = características, sin y	Descubrir estructura, grupos o anomalías	Explorar una ciudad sin mapa
Aprendizaje por Refuerzo	Prueba y error. Recibe recompensas/castigos por sus acciones	Entorno + función de recompensa. No necesita datos históricos etiquetados	Maximizar recompensa acumulada a largo plazo	Entrenar a un perro con premios

1. Aprendizaje Supervisado

Aprendes a partir de ejemplos donde ya conoces la respuesta correcta. El modelo minimiza el error entre su predicción y la etiqueta real.

Caso de uso real: Detección de fraude bancario

Bancos como BBVA o Nubank usan modelos supervisados para marcar transacciones como "fraude" o "legítima".

- **Datos requeridos:** Histórico de millones de transacciones donde cada una está etiquetada como `fraude=1` o `legítima=0`. Cada transacción tiene features: monto, país, hora, dispositivo, tipo de comercio.
- **Objetivo:** Clasificación binaria. Dado una nueva transacción, predecir la probabilidad de que sea fraude y bloquearla en tiempo real si supera un umbral.
- **Algoritmos típicos:** Regresión logística, Random Forest, XGBoost, Redes neuronales.

2. Aprendizaje No Supervisado

Aquí no hay "respuesta correcta". El algoritmo debe encontrar estructura en los datos por su cuenta.

Caso de uso real: Segmentación de clientes en e-commerce

Amazon, Mercado Libre o Netflix lo usan para agrupar usuarios con comportamientos similares sin definir los grupos de antemano.

- **Datos requeridos:** Comportamiento de usuarios: productos vistos, categorías compradas, frecuencia de compra, ticket promedio, hora de conexión. Todo sin etiquetas.
- **Objetivo:** Clustering. Descubrir segmentos naturales como "Compradores nocturnos de tecnología", "Cazadores de ofertas de moda", "Clientes inactivos". Con eso se personalizan campañas y recomendaciones.
- **Algoritmos típicos:** K-means, DBSCAN, PCA para reducción de dimensionalidad, Autoencoders.

3. Aprendizaje por Refuerzo

Un "agente" interactúa con un entorno, toma acciones y aprende de las consecuencias. No le dices qué hacer, solo si lo hizo bien o mal con una recompensa.

Caso de uso real: Optimización de subastas en publicidad digital

Google Ads y Meta Ads usan RL para decidir en tiempo real cuánto pujar por mostrar tu anuncio a un usuario específico.

- **Datos requeridos:** No un dataset estático. Necesita un entorno simulado o real con: estado = perfil del usuario + contexto, acción = cuánto pujar, recompensa = si hubo clic/conversión y a qué costo.

- **Objetivo:** Maximizar ROI a largo plazo. El agente aprende una política: "si el usuario es de CDMX, 25 años, buscó vuelos, entonces pujar \$12 MXN". Si se pasa de presupuesto se castiga, si convierte se premia.
- **Algoritmos típicos:** Q-Learning, DQN, PPO, Bandidos multi-brazo.

Regla rápida para elegir:

1. ¿Tienes la respuesta que quieres predecir? → **Supervisado**
2. ¿Solo quieres entender/explorar tus datos? → **No supervisado**
3. ¿Necesitas tomar secuencias de decisiones para optimizar algo a futuro? → **Refuerzo**

Conclusión: Es importante distinguir el enfoque de nuestra investigación, los datos que poseemos y el flujo de trabajo deseado para poder tomar una decisión certera sobre el tipo de herramienta de IA que se usará.

Prompt 2. ¿Qué es una red neuronal artificial y cómo se relaciona con el funcionamiento del cerebro humano? Explica qué son las épocas de entrenamiento, la función de pérdida y el descenso de gradiente, y cómo estos tres conceptos trabajan juntos para que un modelo "aprenda".

Una **red neuronal artificial** (RNA) es un modelo de computadora inspirado en cómo creemos que funciona el cerebro. Pero ojo: "inspirado" ≠ "idéntico". Te explico las dos partes y luego cómo aprende.

1. RNA vs cerebro humano: parecidos y diferencias

	Cerebro humano	Red neuronal artificial
Unidad básica	Neurona biológica: recibe señales químicas/eléctricas por dendritas, y si supera un umbral, dispara por el axón	Neurona artificial: recibe números, hace suma ponderada + sesgo , pasa por una función de activación y da una salida
Conexiones	Sinapsis: se fortalecen o debilitan con la experiencia. Hay ~86 mil millones de neuronas y ~100 billones de sinapsis	Pesos: números que multiplican cada entrada. Se ajustan con matemáticas. GPT-4 tiene ~1.7 billones de parámetros

	Cerebro humano	Red neuronal artificial
Procesamiento	Paralelo, analógico, bioquímico, muy eficiente en energía. Aprende con pocos ejemplos	Paralelo, digital, matricial. Necesita toneladas de datos y GPUs que gastan mucha energía
Plasticidad	Hebb: "neuronas que se disparan juntas, se conectan juntas"	Backpropagation + descenso de gradiente: calculamos el error y ajustamos pesos hacia atrás

En resumen: la RNA toma la idea de "muchas unidades simples conectadas que modifican sus conexiones para aprender". El cerebro es mucho más complejo, con química, sueño, emociones y sin función de pérdida explícita.

2. ¿Cómo aprende una RNA? Los 3 conceptos clave

Imagina que la red es un estudiante tratando de adivinar el precio de casas. Tiene que ajustar sus "perillas" internas hasta que sus predicciones se parezcan a la realidad.

Época de entrenamiento

Una época = la red vio todo tu conjunto de datos de entrenamiento 1 vez completa.

Si tienes 10,000 fotos de gatos/perros y haces 5 épocas, la red pasó 5 veces por las 10,000 fotos.

- Pocas épocas: underfitting, no aprendió suficiente
- Demasiadas épocas: overfitting, se memorizó el dataset y no generaliza

Función de pérdida

Es el "profesor" que califica qué tan mal lo hizo la red. Toma la predicción de la red y la respuesta correcta, y escupe un solo número: el error.

Ejemplos:

- **MSE:** $(prediccion - real)^2$ para regresión. Si dijo que la casa vale \$200k y valía \$250k, pérdida = 2500-
- **Cross-entropy:** para clasificación. Castiga mucho cuando dice 99% gato pero era perro

Mientras más bajo el número, mejor. Pérdida = 0 sería predicción perfecta.

Descenso de gradiente

Es el método que usa la red para mejorar. Si la pérdida es una montaña, el gradiente te dice la dirección más empinada hacia arriba. Entonces haces lo contrario: das un pasito hacia abajo.

Paso a paso:

1. **Forward pass:** mete datos → la red predice algo
2. **Calcular pérdida:** compara predicción vs real
3. **Backward pass / backpropagation:** calcula cuánto contribuyó cada peso al error
4. **Actualizar pesos:** $\text{nuevo_peso} = \text{peso_viejo} - \text{tasa_aprendizaje} * \text{gradiente}$

La **tasa de aprendizaje** es qué tan grande es el "pasito". Muy grande y te pasas el mínimo. Muy chica y tardas años.

3. ¿Cómo trabajan juntos los 3?

Piensa en aprender a tirar al aro de básquet:

1. **Época:** Tiras 100 veces al aro. Eso es 1 época. Para mejorar, necesitas varias épocas/temporadas de práctica
2. **Función de pérdida:** Después de cada tiro, mides a cuántos cm quedó la pelota del centro del aro. Esa distancia es tu "pérdida"
3. **Descenso de gradiente:** Tu cerebro ajusta el ángulo del brazo y la fuerza basado en si te pasaste o te quedaste corto. Si la pelota se fue a la izquierda, el próximo tiro apuntas más a la derecha

En código sería así:

Code

```
por cada época:    por cada lote de datos:    1. predicciones = modelo(datos)    # Forward
2. pérdida = función_pérdida(predicciones, reales)    3. gradientes = calcular_gradientes(pérdida)
# Backprop    4. pesos = pesos - lr * gradientes    # Descenso de gradiente
```

1 línea oculta

Después de muchas épocas, la pérdida baja, los pesos se estabilizan, y el modelo "aprendió".

Analogía final con el cerebro:

Cuando aprendes a andar en bici, no tienes una "función de pérdida" matemática. Pero sí tienes dolor al caerte = error alto. Tu cerebelo ajusta qué músculos activas = "actualizar pesos". Después de varios días/épocas, dejas de caerte = pérdida baja. La diferencia es que la RNA necesita millones de "caídas" calculadas matemáticamente, y el cerebro aprende con mucha menos experiencia.

Conclusiones: El entendimiento del funcionamiento de las redes neuronales artificiales (RNA) es fundamental para la creación de

diferentes modelos de deep learning en nuestro flujo de trabajo pues, al buscar simular el comportamiento cerebral, su arquitectura puede ser muy diversa, brindando una amplia gama de configuraciones posibles.

Prompt 3. ¿Cuáles son las principales diferencias entre Scikit-learn y TensorFlow como frameworks de Machine Learning? ¿En qué situaciones es más conveniente usar cada uno, y existe algún caso en que sea recomendable utilizarlos juntos en un mismo proyecto?

Scikit-learn vs TensorFlow: ¿Cuál usar y cuándo?

Ambos son librerías de Machine Learning para Python, pero apuntan a problemas y niveles de complejidad muy distintos. Piensa en ellos como "caja de herramientas básica" vs "fábrica industrial".

	Scikit-learn	TensorFlow
Tipo de ML	Machine Learning clásico/tradicional	Deep Learning + ML clásico
Enfoque principal	Algoritmos estadísticos listos para usar: regresión lineal, SVM, Random Forest, K-means, PCA	Redes neuronales, tensores, computación en GPU/TPU. Modelos complejos a gran escala
Curva de aprendizaje	Baja. API muy consistente: <code>.fit()</code> , <code>.predict()</code> , <code>.transform()</code>	Media-alta. Conceptos de grafos, capas, tensores, optimizadores, epochs
Datos que maneja mejor	Datos tabulares/estructurados. Datasets pequeños-medianos, <1M filas	Imágenes, texto, audio, video, series temporales. Datasets masivos
Hardware	Solo CPU. No necesita GPU	Optimizado para GPU/TPU.

	Scikit-learn	TensorFlow
		Entrenar LLMs sin GPU es inviable
Flexibilidad del modelo	Baja. Usas algoritmos predefinidos. Difícil crear arquitecturas nuevas	Muy alta. Construyes la arquitectura capa por capa con Keras API
Preprocesamiento	Incluye muchas herramientas: <code>StandardScaler</code> , <code>OneHotEncoder</code> , <code>Pipeline</code>	Espera que los datos ya lleguen como tensores. Keras tiene capas de preprocesamiento pero sklearn es más completo aquí
Despliegue	Modelo = archivo .pkl. Fácil de poner en producción para APIs	TensorFlow Serving, TensorFlow Lite, TensorFlow.js. Optimizado para móvil, web, edge

Diferencia clave en 1 línea:

Scikit-learn es para cuando ya sabes qué algoritmo usar y tus datos caben en RAM.
TensorFlow es para cuando necesitas inventar el algoritmo y entrenar por días en GPU.

¿Cuándo conviene usar cada uno?

Usa Scikit-learn si:

1. **Tu problema es tabular:** Predicción de precios, churn de clientes, scoring crediticio, clasificación con CSV/Excel.
2. **Necesitas un baseline rápido:** En 5 líneas tienes un RandomForest entrenado. Perfecto para POCs y hackathons.
3. **Interpretabilidad importa:** Necesitas `feature_importances_`, SHAP, o explicar por qué el modelo decidió X. Los árboles y regresiones lineales son mucho más explicables que una red neuronal.
4. **Dataset pequeño-mediano:** <100k filas. Con más datos, los modelos clásicos se estancan y no mejoran.

5. **No tienes GPU:** Todo corre en CPU de laptop sin problema.

Ejemplos: Predecir si un cliente pagará un crédito, segmentar usuarios con K-means, detectar fraude con datos transaccionales.

Usa TensorFlow/Keras si:

1. **Trabajas con datos no estructurados:** Visión por computadora, NLP, audio, series temporales complejas.
2. **Necesitas Deep Learning:** CNNs para imágenes, RNNs/LSTMs/Transformers para texto, Autoencoders, GANs.
3. **Tienes muchos datos:** >1M ejemplos. Las redes neuronales siguen mejorando con más datos donde RandomForest se estanca.
4. **Quieres arquitecturas custom:** Combinar CNN + LSTM, usar attention, fine-tunear BERT/GPT.
5. **Despliegue especializado:** Necesitas correr el modelo en Android con TFLite, en browser con TF.js, o servir millones de requests/día.

Ejemplos: Clasificar imágenes médicas, traducir texto, generar descripciones, detectar objetos en video, LLMs.

¿Usarlos juntos en el mismo proyecto? Sí, y es súper común

No son excluyentes. El flujo más típico es: **Scikit-learn para preprocesamiento + TensorFlow para modelado**, o viceversa.

Casos reales donde se combinan:

1. Pipeline híbrido de features

Usas `sklearn.preprocessing` y `sklearn.feature_extraction` para limpiar datos, hacer `TfidfVectorizer` o `PCA`, y luego le pasas esos vectores a una red neuronal en TensorFlow.

`sklearn` tiene el mejor `ColumnTransformer` y `Pipeline` para datos mixtos numéricos/categoricos.

2. Ensamblés Stacking

Entrenas un modelo de TensorFlow + varios de sklearn: XGBoost, RandomForest, SVM. Luego usas `sklearn.ensemble.StackingClassifier` para que un metamodelo combine las predicciones de todos. Esto gana muchas competencias de Kaggle.

3. Validación y métricas

Entrenas la red en TF, pero usas `sklearn.metrics` para calcular `classification_report`, `roc_auc_score`, `confusion_matrix`. También `sklearn.model_selection` para `KFold`, `StratifiedKFold`, `GridSearchCV` aunque el estimador sea de Keras via `KerasClassifier`.

4. AutoML + Deep Learning

Usas sklearn para hacer selección de features con `RFE` o `SelectKBest`, reducir dimensionalidad, y solo las mejores 50 features entran a la red de TF. Ahorras mucho tiempo de entrenamiento.

5. Transfer Learning + ML clásico

Tomas una red preentrenada en TF como ResNet50, le quitas la última capa y usas la salida como "embeddings" de 2048 dimensiones. Luego entrenas un `LogisticRegression` o `SVC` de sklearn sobre esos embeddings. Mucho más rápido que fine-tunear toda la red y funciona genial con pocos datos.

Regla práctica para decidir:

1. Empieza siempre con sklearn. Si con 1 hora logras 92% accuracy, ¿para qué complicarte con TF?
2. Si tus datos son imágenes/texto/audio o sklearn se queda en 70% y tienes más datos → salta a TensorFlow.
3. Si usas TF, igual ten sklearn instalado. Lo vas a usar para métricas, splits y preprocesamiento.

Conclusiones: Scikit-Learn resulta ser una paquetería que podría considerarse una primera aproximación a la resolución de la problemática, siempre y cuando ya se conozca el algoritmo que se usará y si los datos caben en la memoria RAM, por otro lado Tensorflow puede tomarse como una paquetería que permite configurar la arquitectura desde sus bases y adaptarla a nuestras necesidades.